SENG 300 Leaderboard System

The purpose of this document is to describe the design, architecture, and behaviour of the leaderboard subsystem. This is responsible for storing, retrieving, and displaying player scores for supported games. The leaderboard subsystem operates using a client-to-server model, in which player actions are initiated through the game system, forwarded to the leaderboard client layer, processed by the leaderboard server, and persisted in a database.

This document provides:

- Subsystem responsibilities
- System architecture
- Use cases and behaviour
- Sequence flows
- Assumptions and scope decisions

Scope

This iteration one explicitly models leaderboard functionality, which includes:

- Viewing leaderboard rankings
- Viewing personal score
- Submitting scores
- Score validation
- Options filtering

Out of scope (for now):

- Matchmaking logic
- Ranking algorithms
- MMR calculations (shown as an external dependency in this iteration)

Actors

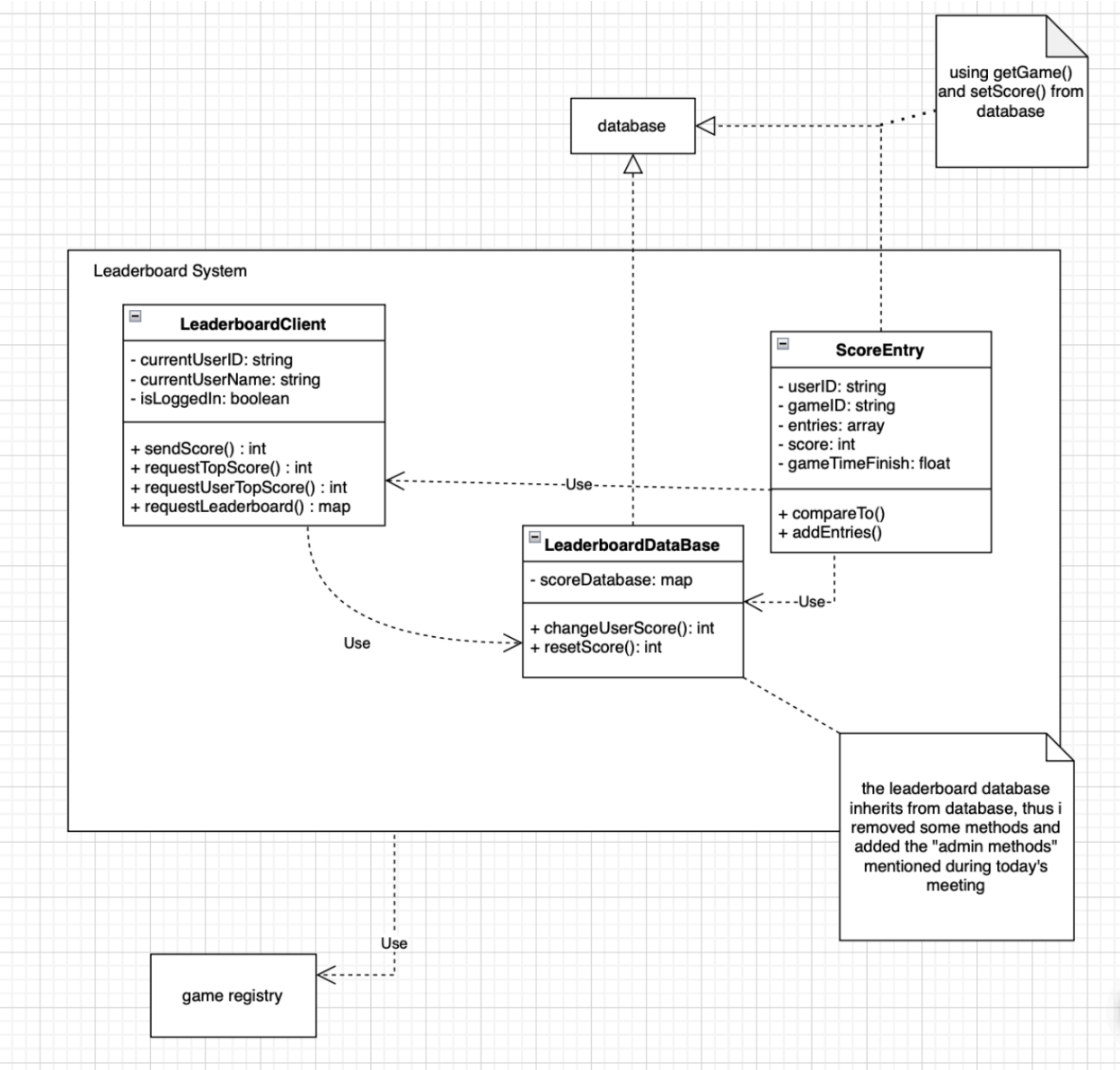
Player (Primary Actor):
  Initiates leaderboard interactions
Leaderboard System Actor (Game Server) (Secondary Actor):
  Stores, retrieves, and updates leaderboard data.

Architecture Overview

This leaderboard subsystem follows a layered flow of:

Player → Game System → LeaderboardClient → LeaderboardServer → LeaderboardDatabase

Here is our class diagram (provided by Jaspreet) **with notes on it**



Layer Responsibilities

Game System:

- Detects game completion
- Calculates final scores
- Initiates leaderboard requests

LeaderboardClient:

- Maintains the user session state
- Sends requests to the server
- Receives results for display

LeaderboardServer:

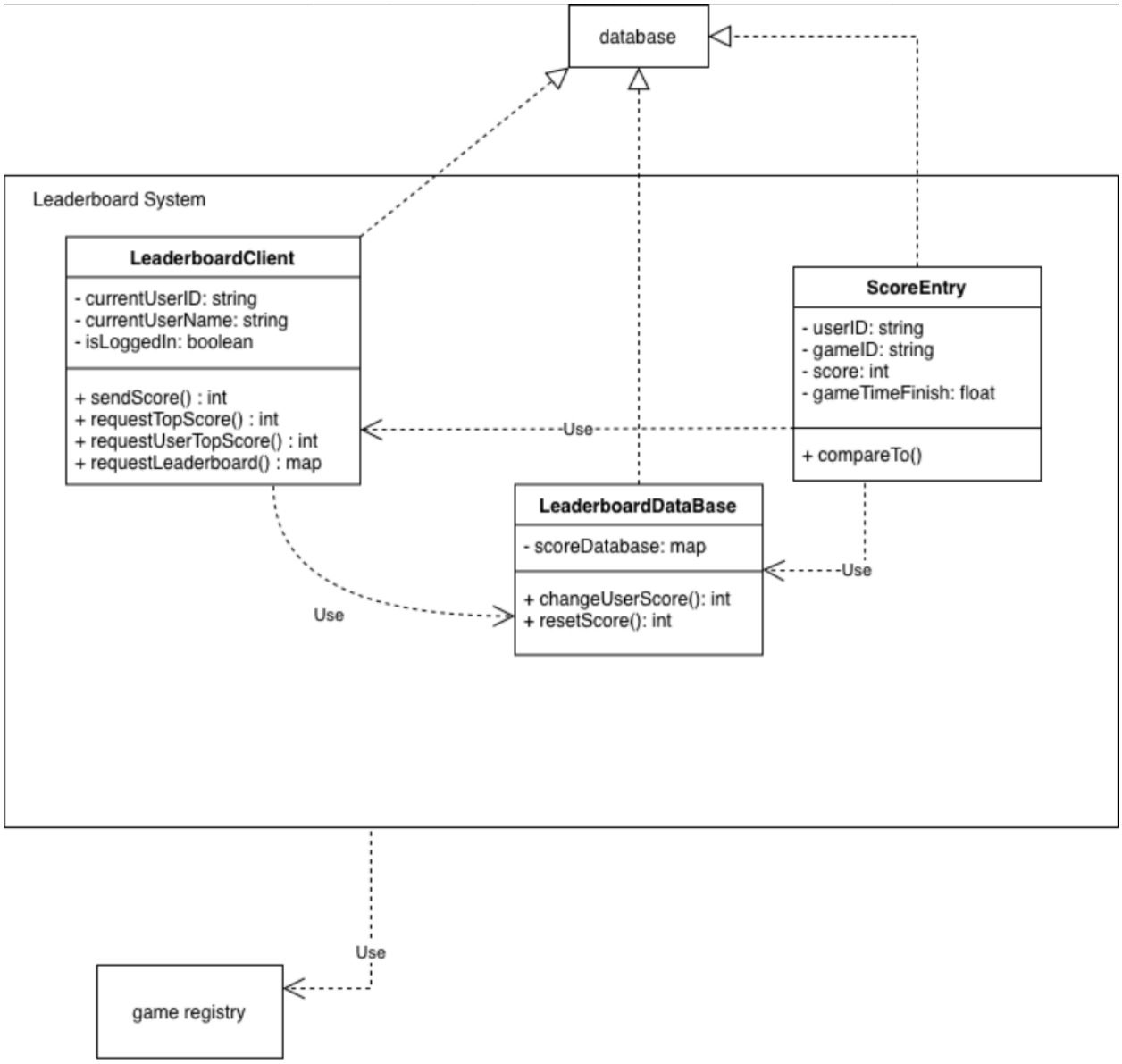
- Handles validation
- Processes leaderboard queries
- Stores score entries

LeaderboardDatabase:

- Stores ScoreEntry objects
- Provides sorted leaderboard data

Class Design Overview

The class diagram models the core data and logical separation.



LeaderboardClient

This represents the local interface between the game system and leaderboard services

Attributes:

- currentUserID
- currentUserName
- isLoggedIn

Responsibilities:

- sendScore()
- requestTopScore()
- requestUserTopScore()
- requestLeaderboard()

LeaderboardServer

This represents the server-side processing layer.

Responsibilities:

- serScore()
- getTopScores()
- getUserTopScore()
- handleClient()

This layer performs validation and coordinates database interactions.

LeaderboardDatabase

This represents score storage abstraction.

Responsibilities:

- changeUserScore()
- resetScore()

ScoreEntry

This represents a single leaderboard record

Attributes:

- userID
- gameID
- Score
- gameTimeFinish

Responsibilities:

- compareTo()

Relationships

LeaderboardClient → uses → LeaderboardServer.
LeaderboardServer → uses → LeaderboardDatabase.
LeaderboardDatabase stores lots of ScoreEntry objects.
ScoreEntry is simply a data entity, not explicitly owned by the client, so it is not aggregated from the client.

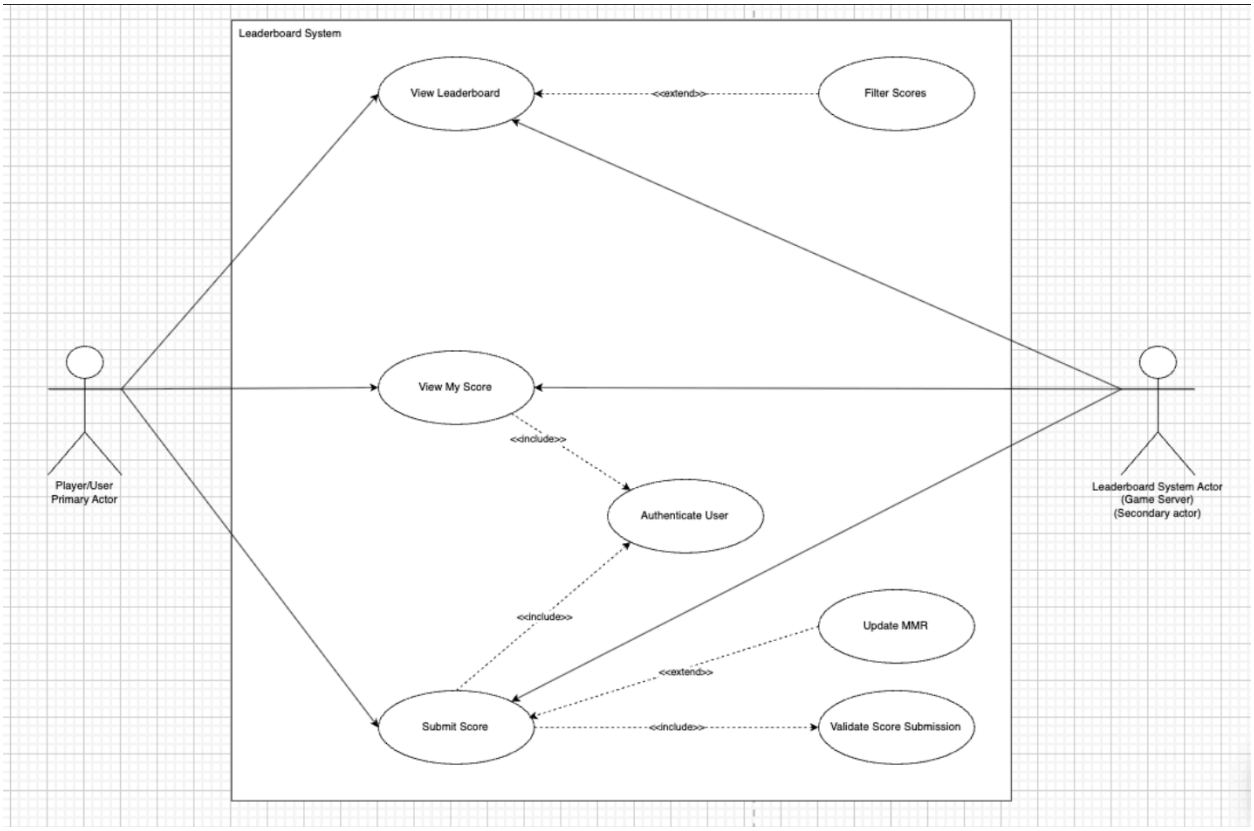
Multiplicity Decisions

- One leaderboardDatabase → many ScoreEntry
- One leaderboardClient → many requests (not explicit ownership)
- Many clients can exist at the same time (concurrently)

Use Case Overview

The use case diagram models the primary user interactions.

Here is the diagram (provided by Sajan):



Core use cases

- View Leaderboard
- View My Score
- Submit Score

Supporting use cases:

- Authenticate User (<<include>>)
- Validate Score Submission (<<include>>)
- Filter Score (<<extend>>)
- Update MMR (<<extend>> | external idea)

Design Rationale

- <<include>> | Required Behaviours:
    - Authenticate User
    - Validate Score Submission
- <<extend>> | Optional Behaviours:
    - Filtering
    - MMR updating

Assumptions:

Here are assumptions for the leaderboard system:

- Sores are stored server-wide
- Each score belongs to a specific userID and gameID
- Leaderboards are sorted primarily by score
- Filtering is applied after retrieval
- Authentication state is maintained client-side

Future Work

Here are some future work/ideas:

- Matchmaking integration
- Advanced ranking logic
- Real-time leaderboard updates
- Multi-game aggregation
- Caching strategies